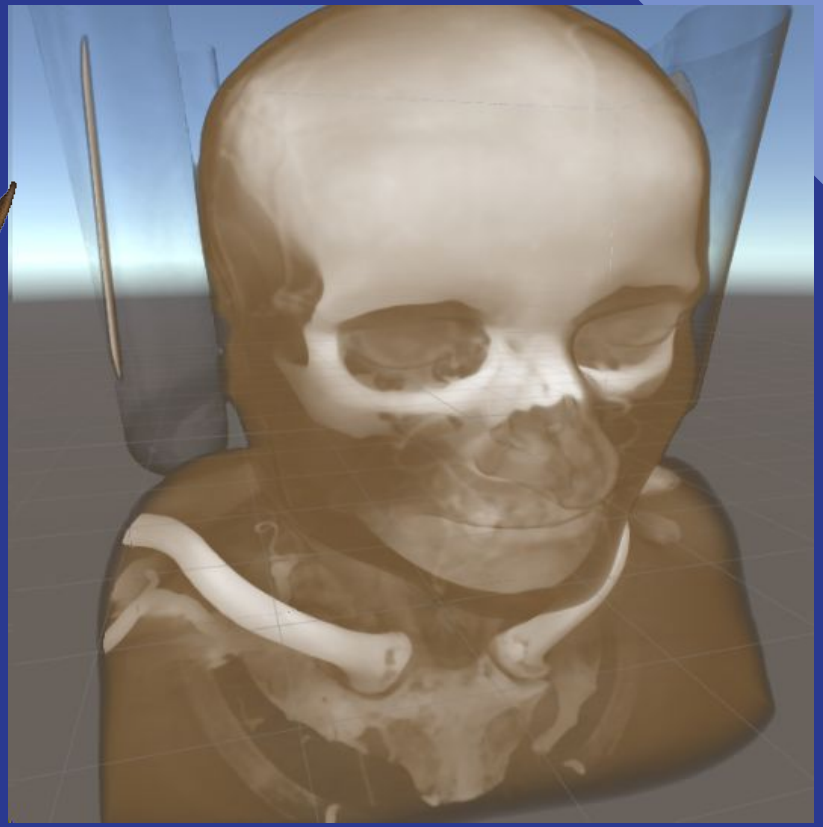
Dimension10

Volume Rendering

In Unity3D



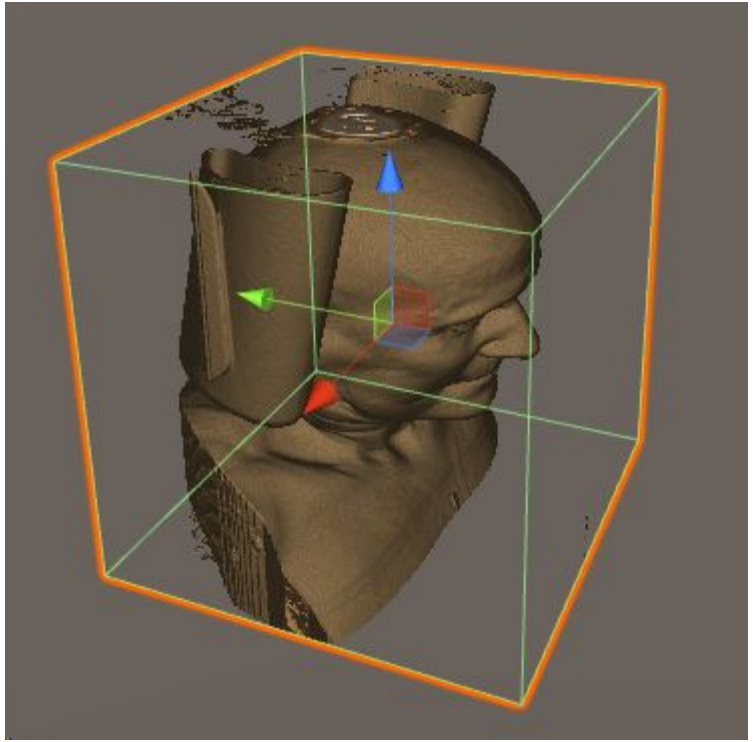
Matias Lavik

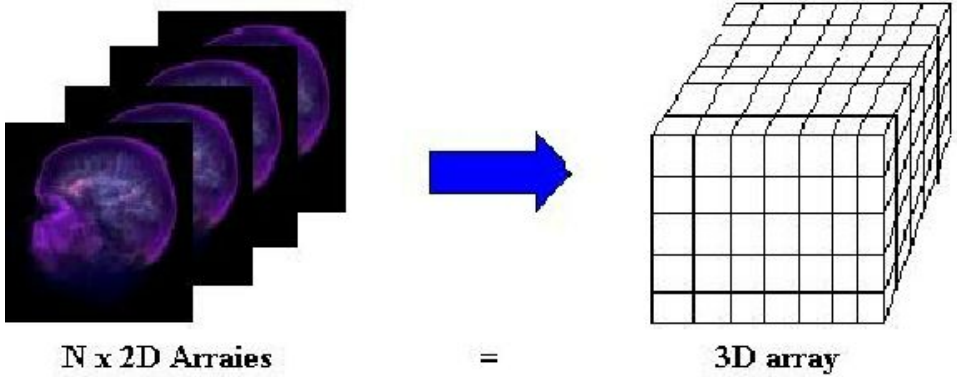
The presentation

- An introduction to volume rendering of medical 3D volume data.
- Based on my hobby-project: https://github.com/mlavik1/UnityVolumeRendering (still WIP)
- What we will cover:
  - Raymarching volume data
  - Techniques for volume rendering: Direct volume rendering with compositing, maximum intensity projection, isosurface rendering
  - Diffuse lighting
  - Transfer functions
  - Multidimensional transfer functions

The data

- We use 3D voxel data from a medical CT scan.
- The value of each data point represents the density.
- The data is read from a file and stored in a 3D texture
- The data fits nicely into a box, so we render a box (we only draw the back faces) and use *raymarching* to fetch the densities of each visible voxel inside the box.





(above illustration by: Klaus D Tönnies https://www.researchgate.net/figure/3D-volume-ata-representation_fig1_238687132)

Raymarching

Raytracing: Cast a ray from the eye through each pixel and find the first triangle(s) hit by the ray. Then draw the colour of that triangle .

Usage: Rendering triangular 3D models (discrete)

Raymarching: Create a ray from/towards the camera. Divide the ray into N steps/samples, and combine the value at each step.

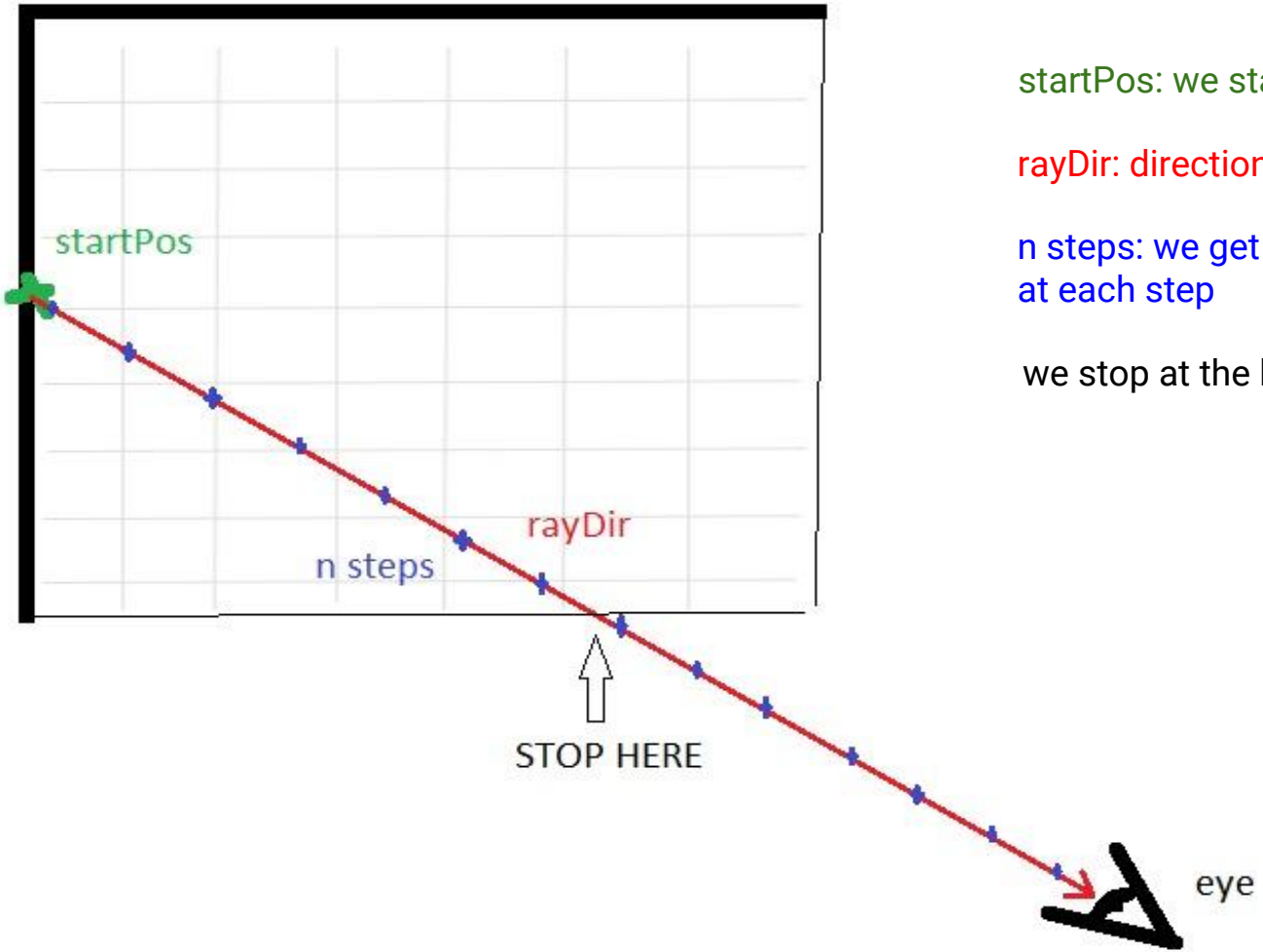
Usage: Rendering *continous* data, such as mathematical equations.

Rendering volume data (such as CT scans)

For CT scans we want to see both the skin, blood veins and bones at the same time - ideally with different colours

Raymarching the data

1. Draw the back-faces of a box
2. For each vertex:
    - 2.1. Set startPos as the vertex local position
    - 2.2. Set rayDir as the direction towards the eye (in model space)
    - 2.3. Divide the ray into N steps/samples
    - 2.4. For each step (starting at startPos moving along rayDir)
        - 2.4.1. Get the density: use currentPos as a texture coordinate and fetch the value from the 3D texture
        - 2.4.2. ??? (use the density to decide the output colour of the fragment/pixel)



startPos: we start from here

rayDir: direction towards the eye

n steps: we get the density (data value) at each step

we stop at the border of the box

```
        // Direct Volume Rendering
        fixed4 frag_dvr (v2f i)
        {
#define NUM_STEPS 1024

            const float stepSize = 1.732f/*greatest distance in box*/ / NUM_STEPS;

            float3 rayStartPos = i.vertexLocal + float3(0.5f, 0.5f, 0.5f);
            float3 rayDir = normalize(ObjSpaceViewDir(float4(i.vertexLocal, 0.0f)));

            float4 col = float4(0.0f, 0.0f, 0.0f, 0.0f);
            for (uint iStep = 0; iStep < NUM_STEPS; iStep++)
            {
                const float t = iStep * stepSize                    ;
                const float3 currPos = rayStartPos + rayDir * t;

                if (currPos.x < 0.0f || currPos.x >= 1.0f || currPos.y < 0.0f || currPos.y > 1.0f || currPos.z < 0.0f || currPos.z > 1.0f)
                    break;

                const float density = getDensity(currPos);

                // TODO: modify "col" based on the density
            }

            col.rgb = col.rgb;

            return col;
        }
```

We will look at 3 implementations:

1. Maximum intensity projection
2. Direct volume rendering with compositing
3. Isosurface rendering

1. Maximum Intensity Projection

- Draw the voxel (data value) with the highest density along the ray.

- Set the output colour to be white with alpha = maxDensity

- This is a simple but powerful visualisation, that highlights bones and big variations in density.

```
fixed4 frag_dvr (v2f i)
{
#define NUM_STEPS 1024

    const float stepSize = 1.732f/*greatest distance in box*/ / NUM_STEPS;

    float3 rayStartPos = i.vertexLocal + float3(0.5f, 0.5f, 0.5f);
    float3 rayDir = normalize(ObjSpaceViewDir(float4(i.vertexLocal, 0.0f)));

    float4 col = float4(0.0f, 0.0f, 0.0f, 0.0f);
    float maxDensity = 0.0f;
    for (uint iStep = 0; iStep < NUM_STEPS; iStep++)
    {
        const float t = iStep * stepSize                    ;
        const float3 currPos = rayStartPos + rayDir * t;

        if (currPos.x < 0.0f || currPos.x >= 1.0f || currPos.y < 0.0f || currPos.y > 1.0f || currPos.z < 0.0f || currPos.z > 1.0f)
            break;

        const float density = getDensity(currPos);
        maxDensity = max(density, maxDensity);
    }
    col = float4(1.0f, 1.0f, 1.0f, maxDensity);

    return col;
}
```

Result

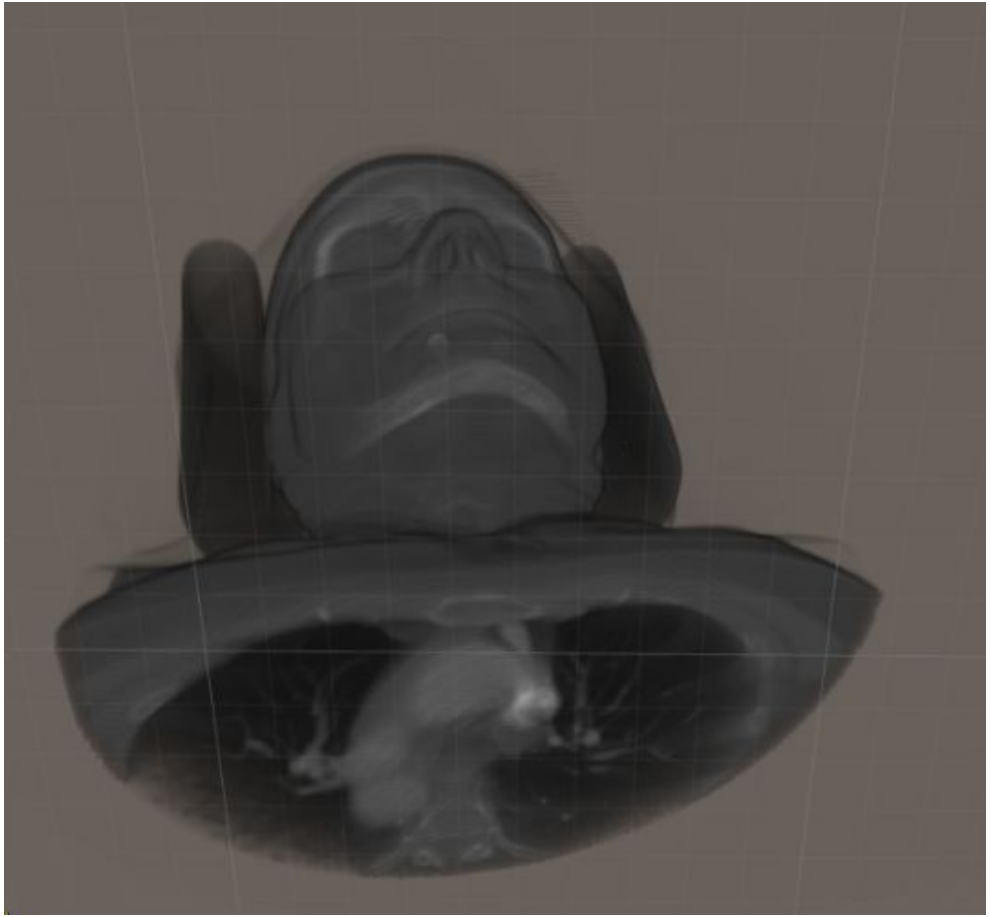
2. Direct Volume Rendering with compositing

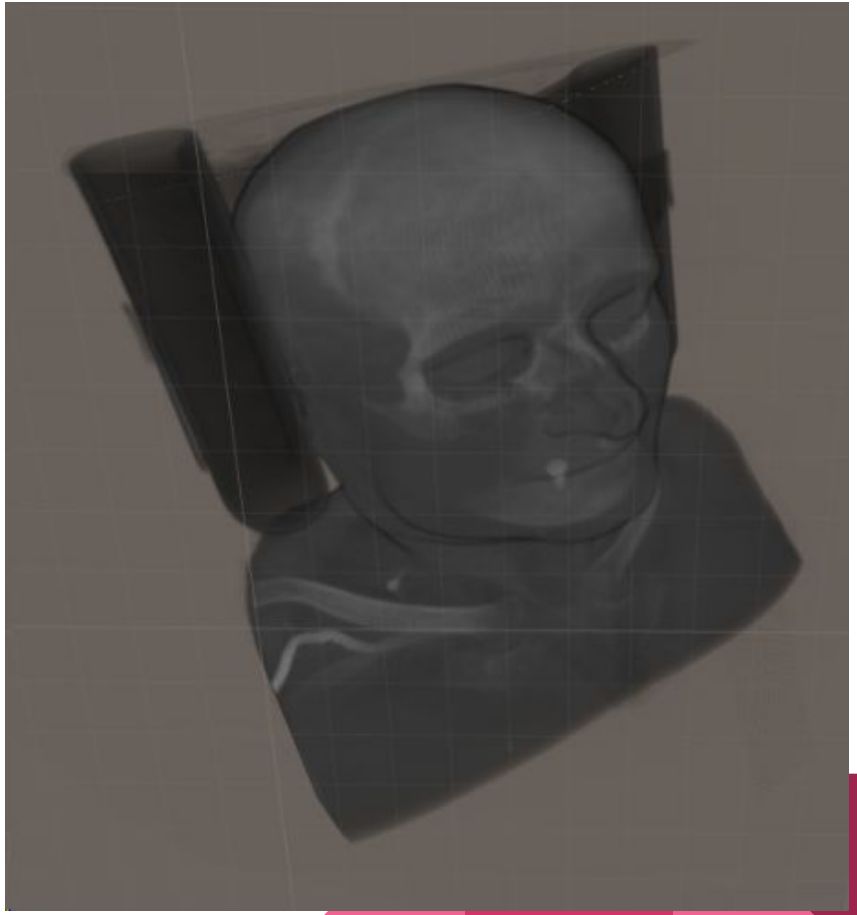
- On each step, combine the current voxel colour with the accumulated colour value from the previous steps.
- We linearly interpolate the RGB values
  - newRGB = lerp(oldRGB, currRGB, currAlpha)
- And add the new alpha to the old alpha multiplied by (1 - new alpha)

```
col.rgb = src.a * src.rgb + (1.0f - src.a)*col.rgb;
col.a = src.a + (1.0f - src.a)*col.a;
```

```
fixed4 frag_dvr (v2f i)
{
#define NUM_STEPS 1024

    const float stepSize = 1.732f/*greatest distance in box*/ / NUM_STEPS;

    float3 rayStartPos = i.vertexLocal + float3(0.5f, 0.5f, 0.5f);
    float3 rayDir = normalize(ObjSpaceViewDir(float4(i.vertexLocal, 0.0f)));

    float4 col = float4(0.0f, 0.0f, 0.0f, 0.0f);
    for (uint iStep = 0; iStep < NUM_STEPS; iStep++)
    {
        const float t = iStep * stepSize                    ;
        const float3 currPos = rayStartPos + rayDir * t;

        if (currPos.x < 0.0f || currPos.x >= 1.0f || currPos.y < 0.0f || currPos.y > 1.0f || currPos.z < 0.0f || currPos.z > 1.0f)
            break;

        const float density = getDensity(currPos);

        float4 src = float4(density, density, density, density * 0.1f);

        col.rgb = src.a * src.rgb + (1.0f - src.a)*col.rgb;
        col.a = src.a + (1.0f - src.a)*col.a;

        if (col.a > 1.0f)
            break;
    }

    return col;
}
```

Source colour: for now we use the density to decide the RGB and alpha values. We multiply alpha by 0.1 to make more transparent.
As a result, bones (high density) are more visible, and the skin (low density) is almost invisible.
Later we will add colours!





3. Isosurface Rendering

- Draw the first voxel (data value) with a density higher than some threshold.
- Now we need to start raymarching at the eye (or ideally at the point where the ray hits the front face of the box) and move away from the eye.
    - When we hit a voxel with density > threshold we stop, and use that density
-

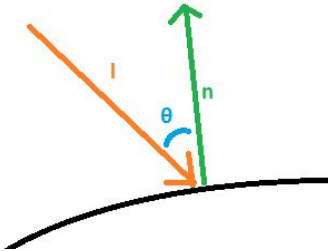
Lighting

- Since we are rendering the surfaces only, we need light reflection to make it look good.

Diffuse reflection:

Light intensity is decided by the angle between the direction to the light source and the surface normal (θ), and a diffuse constant (Kd - we can set it to 1).

$$\boxed{Id = Kd \cdot \cos(\theta)}$$



When l and n are normalised, we have:

$\cos(\theta) = dot(n, l)$

$$\boxed{Id = Kd \cdot dot(n, l)}$$

How do we get the normal?

- Since we are working with 3D voxels and not triangles, getting the normal requires some extra work.

<u>Solution: Use the *gradient*</u>

Gradient = direction of change

Compare the neighbour data values in each axis

```
float xDiff = data[x-1, y, z] - data[x+1, y, z];
float yDiff = data[x, y-1, z] - data[x, y+1, z];
float zDiff = data[x, y, z-1] - data[x, y, z+1];
float3 gradient = (xDiff, yDiff, zDiff);
float3 normal = normalize(gradient);
```

When the data values decrease towards the right (positive X-axis), the normal will point to the right

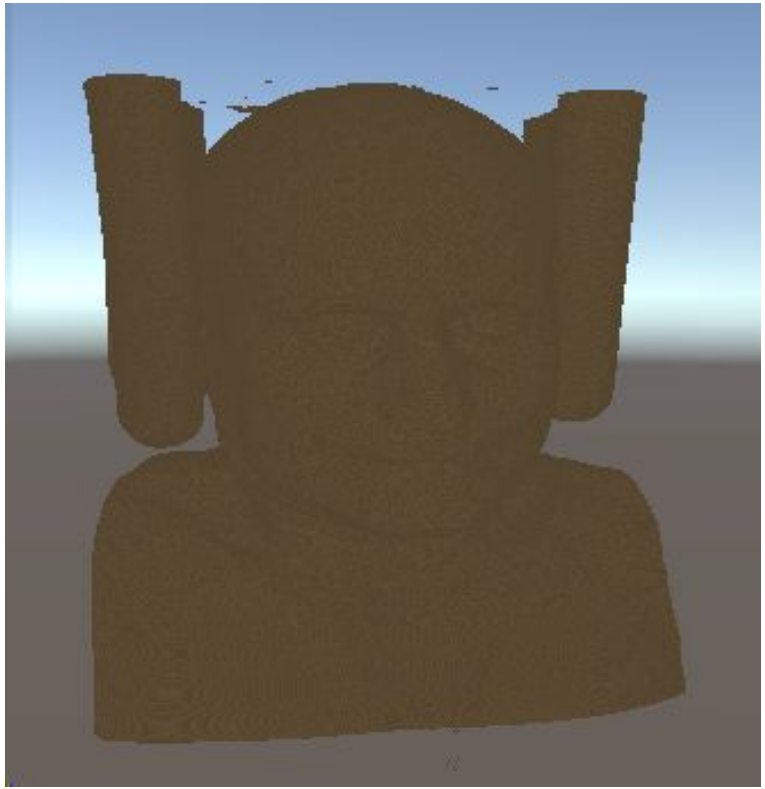
```
fixed4 frag_surf(v2f i)
{
#define NUM_STEPS 1024
    const float stepSize = 1.732f/*greatest distance in box*/ / NUM_STEPS;

    float3 rayStartPos = i.vertexLocal + float3(0.5f, 0.5f, 0.5f);
    float3 rayDir = normalize(ObjSpaceViewDir(float4(i.vertexLocal, 0.0f)));
    // Start from the end, tand trace towards the vertex
    rayStartPos += rayDir * stepSize * NUM_STEPS;
    rayDir = -rayDir;

    float3 lightDir = -rayDir;

    // Create a small random offset in order to remove artifacts
    rayStartPos = rayStartPos + (2.0f * rayDir / NUM_STEPS) * tex2D(_NoiseTex, float2(i.uv.x, i.uv.y)).r;

    float4 col = float4(0,0,0,0);
    for (uint iStep = 0; iStep < NUM_STEPS; iStep++)
    {
        const float t = iStep * stepSize + stepSize * 0.5f;
        const float3 currPos = rayStartPos + rayDir * t;
        // Make sure we are inside the box
        if (currPos.x < 0.0f || currPos.x >= 1.0f || currPos.y < 0.0f || currPos.y > 1.0f || currPos.z < 0.0f || currPos.z > 1.0f)
            continue;

        const float density = getDensity(currPos);
        if (density > _MinVal && density < _MaxVal)
        {
            float3 normal = normalize(getGradient(currPos));
            float lightReflection = dot(normal, lightDir);
            lightReflection =  max(lerp(0.0f, 1.5f, lightReflection), 0.5f);
            col = lightReflection * getTF1DColour(density);
            col.a = 1.0f;
            break;
        }
    }

    return col;
}
```

(Note: we now add a random offset to the ray start position, to improve the sample artifacts.)

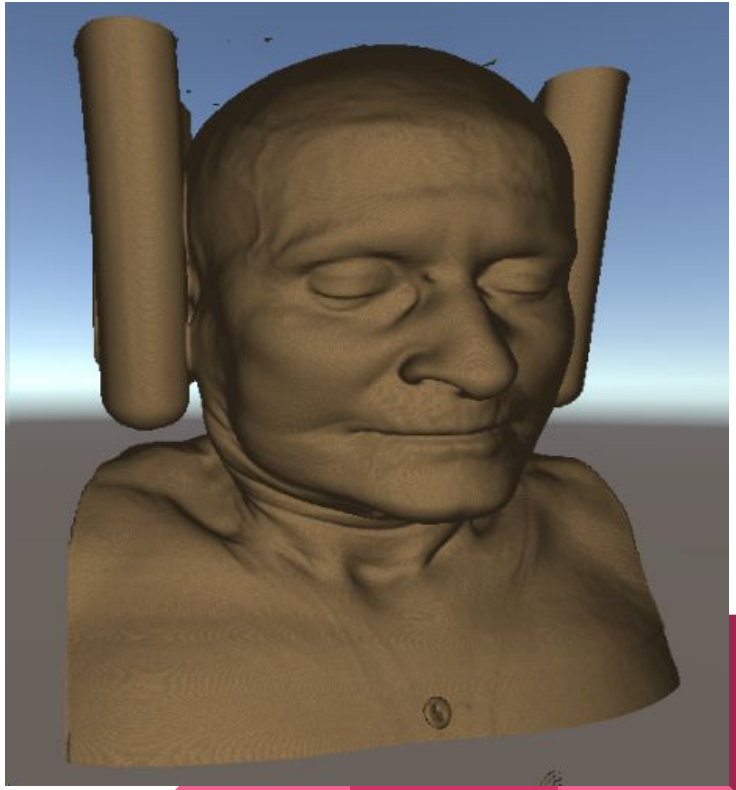
You might want to use another lighting calculation, such as Phong (which includes specular reflection as well).

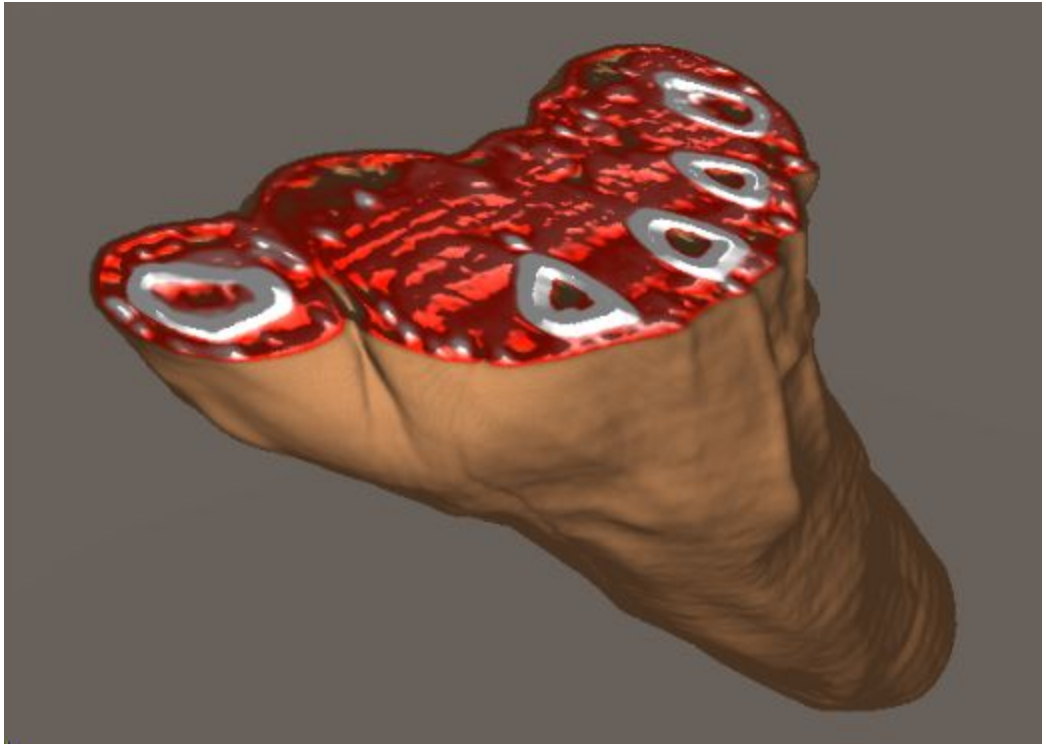
NOTE: Here we use a transfer function to decide colour. (see next slides)

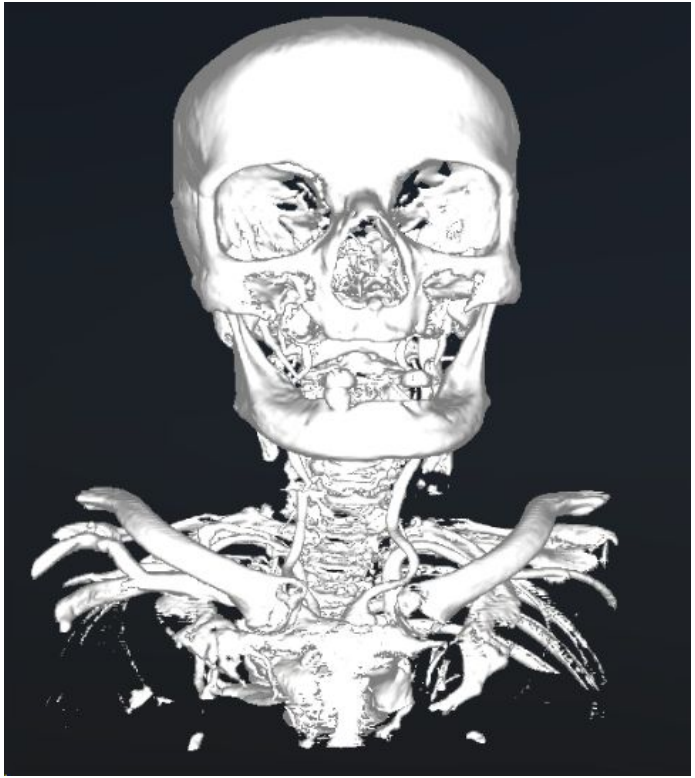
Without lighting



With lighting







With Transfer Function (see next slides)

Transfer Functions

- Basic idea: Use the density to decide colour and opacity/transparency.
  - Example:
    - Bones are white and opaque
    - Blood vessels are red and semi-transparent
    - The skin is brown and almost invisible
- Implementation

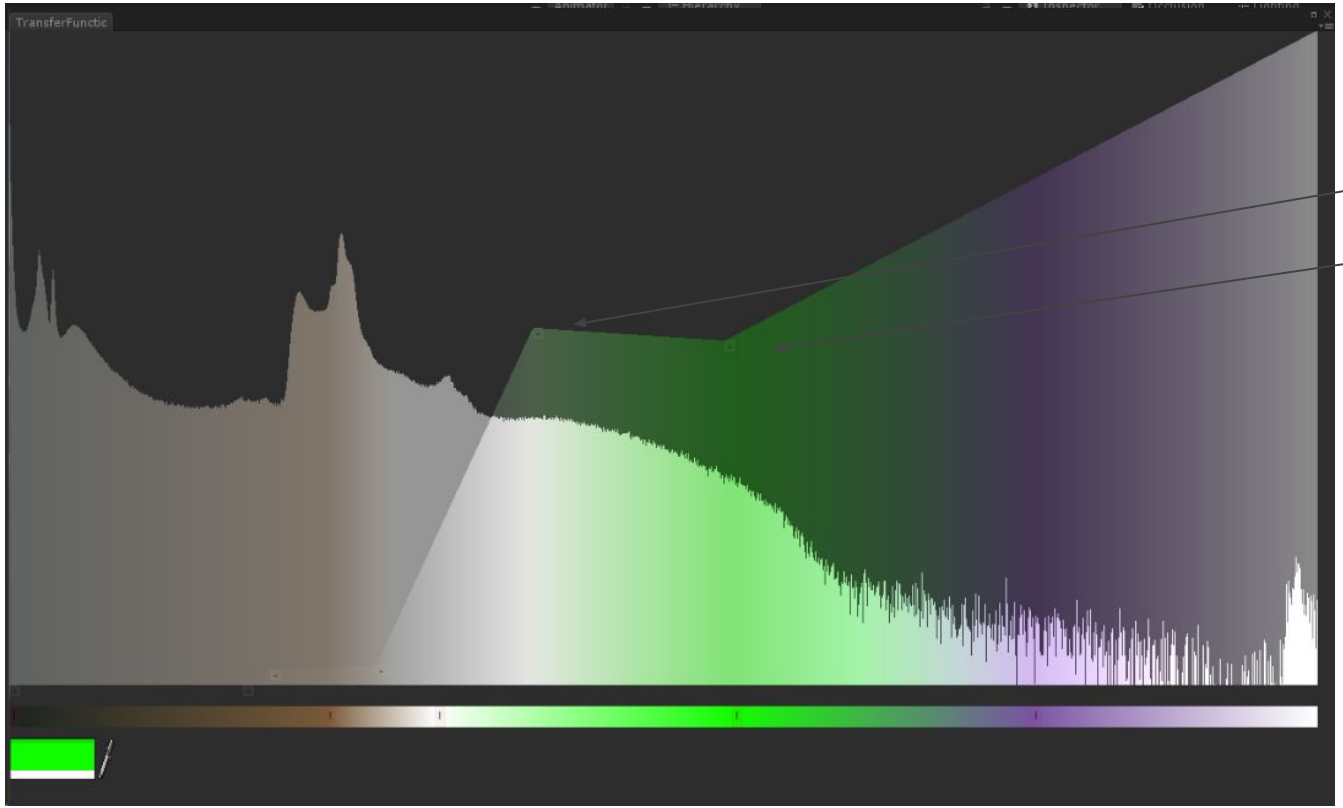
1. Allow the user to add colour knots and alpha knots to a box, where the x-axis represents the density and the y-axis defines the opacity
2. Generate a 1D texture where x-axis = density and the textel (pixel)'s RGBA defines the colour and opacity.
3. Shader: After getting the density of a voxel, use the density to look up the RGBA value in the transfer function texture.

(based on the direct volume rendering shader from previous slides)

```
const float density = getDensity(currPos);

float4 src = getTF1DColour(density);

col.rgb = src.a * src.rgb + (1.0f - src.a)*col.rgb;
col.a = src.a + (1.0f - src.a)*col.a;
```

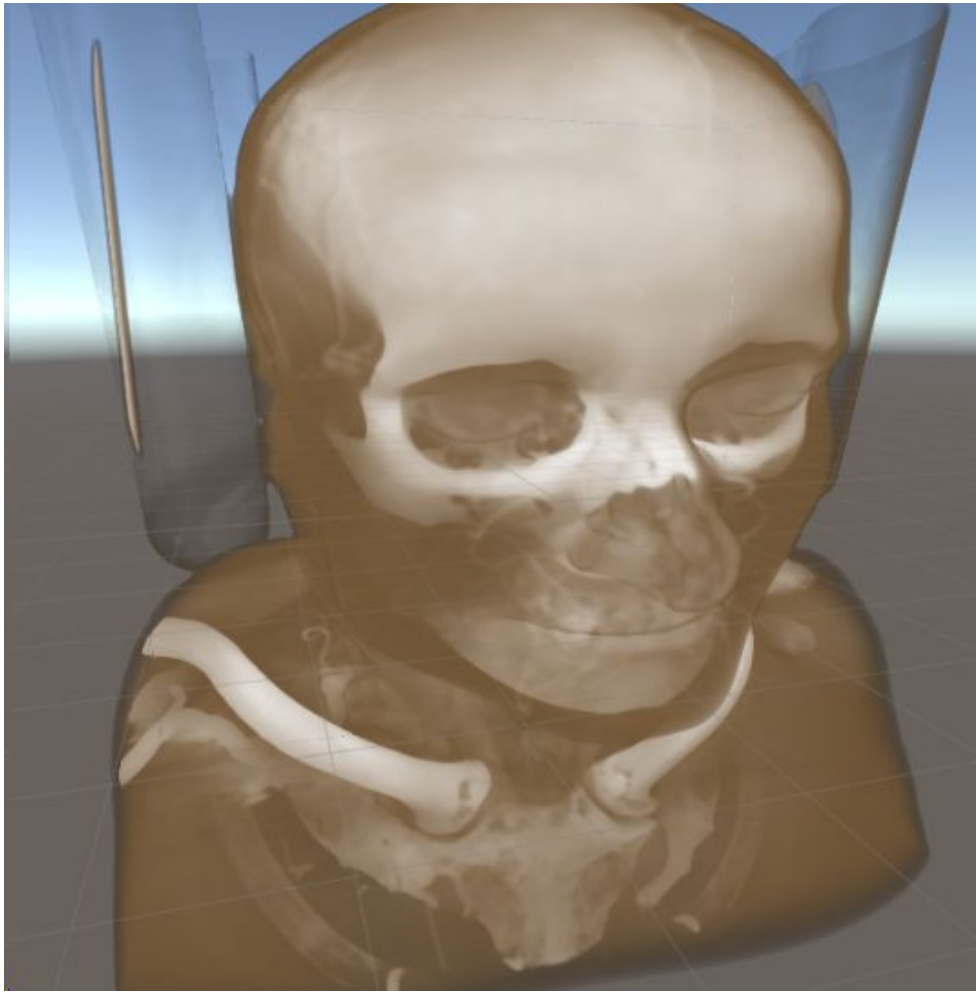
```
// Gets the colour from a 1D Transfer Function (x = density)
float4 getTF1DColour(float density)
{
    return tex2Dlod(_TFTex, float4(density, 0.0f, 0.0f, 0.0f));
}
```

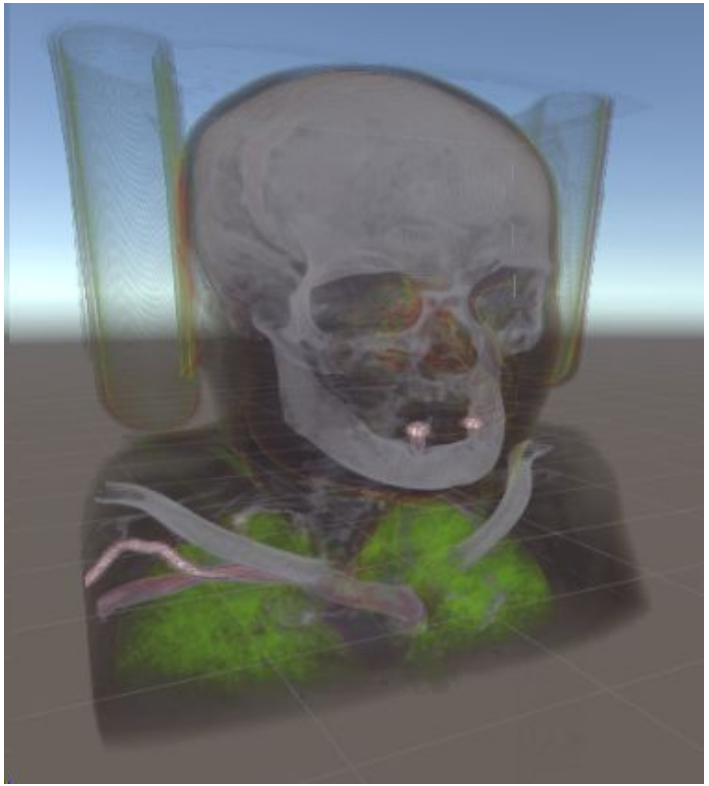
Note: Unity doesn’t support 1D textures so we use a 2D texture with height=1



Alpha knots: move up and down to change the alpha value

X-axis: Density
Y-axis: Opacity (alpha) at density X
Gradient bar: Colour at density X





Multidimensional Transfer Functions

- Sometimes opacity is not enough to identify the parts we want to highlight
  - Different parts of the object might have similar densities
- We need another dimension in our transfer function!

Gradient magnitude

- Gradient = direction of change (already covered this in previous slides)

- Gradient magnitude = magnitude of the gradient vector
  (= how much the density changes at the point - or the “derivative” if you want)

- Using both density and gradient magnitude when setting colours/alphas allows us to highlight surfaces while ignoring what’s beneath it.

```
float3 gradient = getGradient(currPos);
float mag = length(gradient) / 1.75f;
float4 src = getTF2DColour(density, mag);
```

```
// Gets the gradient at the specified position
float3 getGradient(float3 pos)
{
    return tex3Dlod(_DataTex, float4(pos.x, pos.y, pos.z, 0.0f)).rgb;
}
```

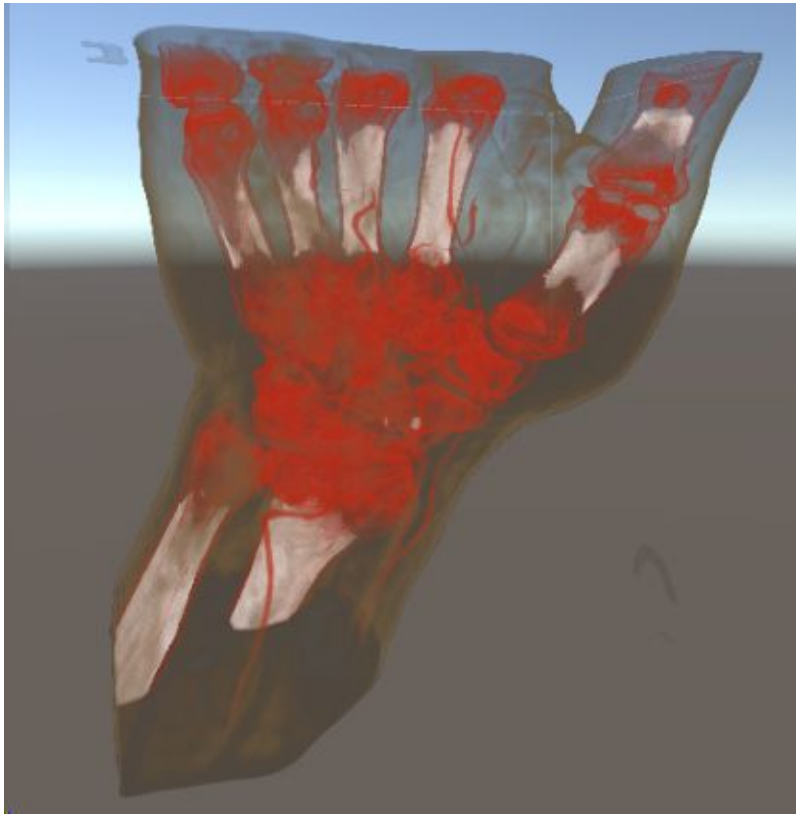
```
// Gets the colour from a 2D Transfer Function (x = density, y = gradient magnitude)
float4 getTF2DColour(float density, float gradientMagnitude)
{
    return tex2Dlod(_TFTex, float4(density, gradientMagnitude, 0.0f, 0.0f));
}
```

NOTE: I store the gradients in the data texture's RGB (and the data value is stored in the A)

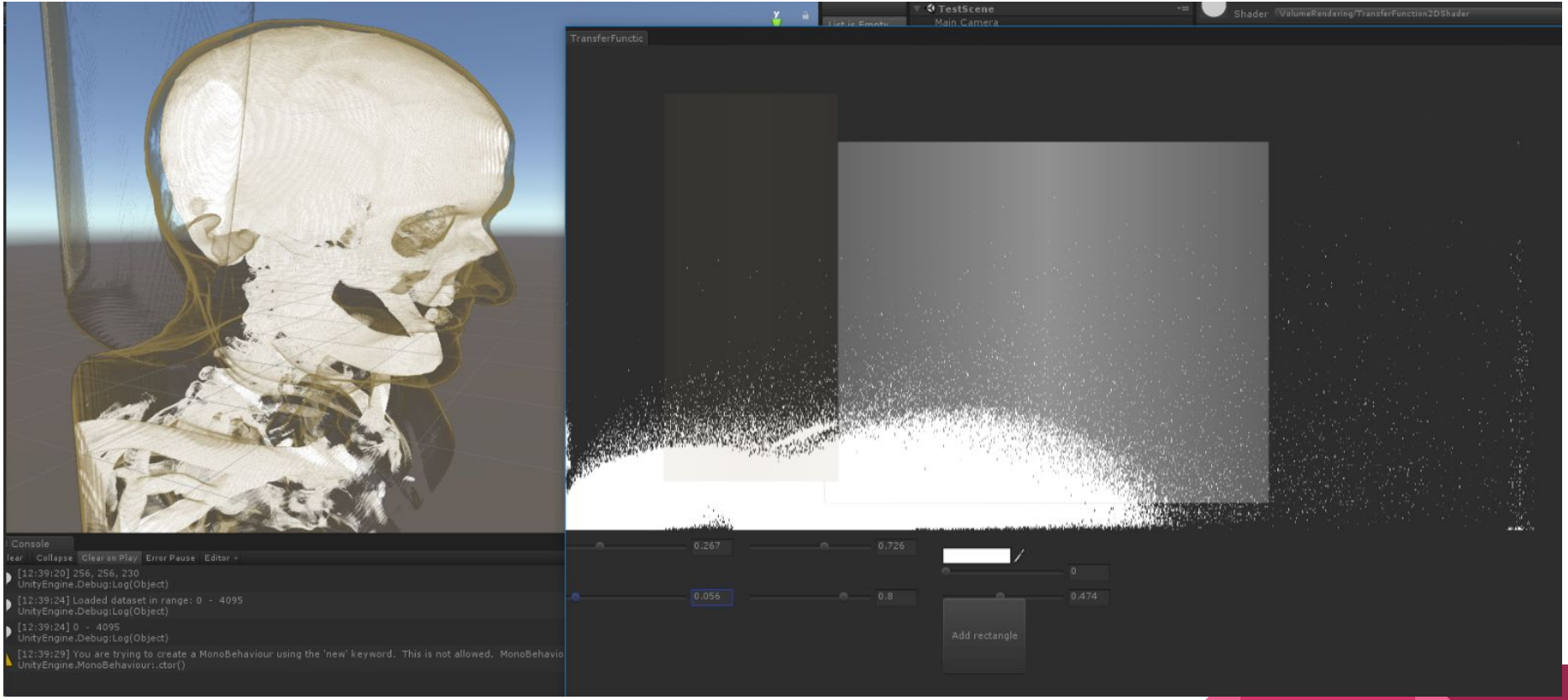
We could calculate the gradients in the shader, but that is expensive. This is a tradeoff, since it uses more memory.

See the earlier slide about "isosurface rendering" for how to calculate gradients, or my GitHub: https://github.com/mlavik1/UnityVolumeRendering

1D Transfer Function



2D Transfer Function

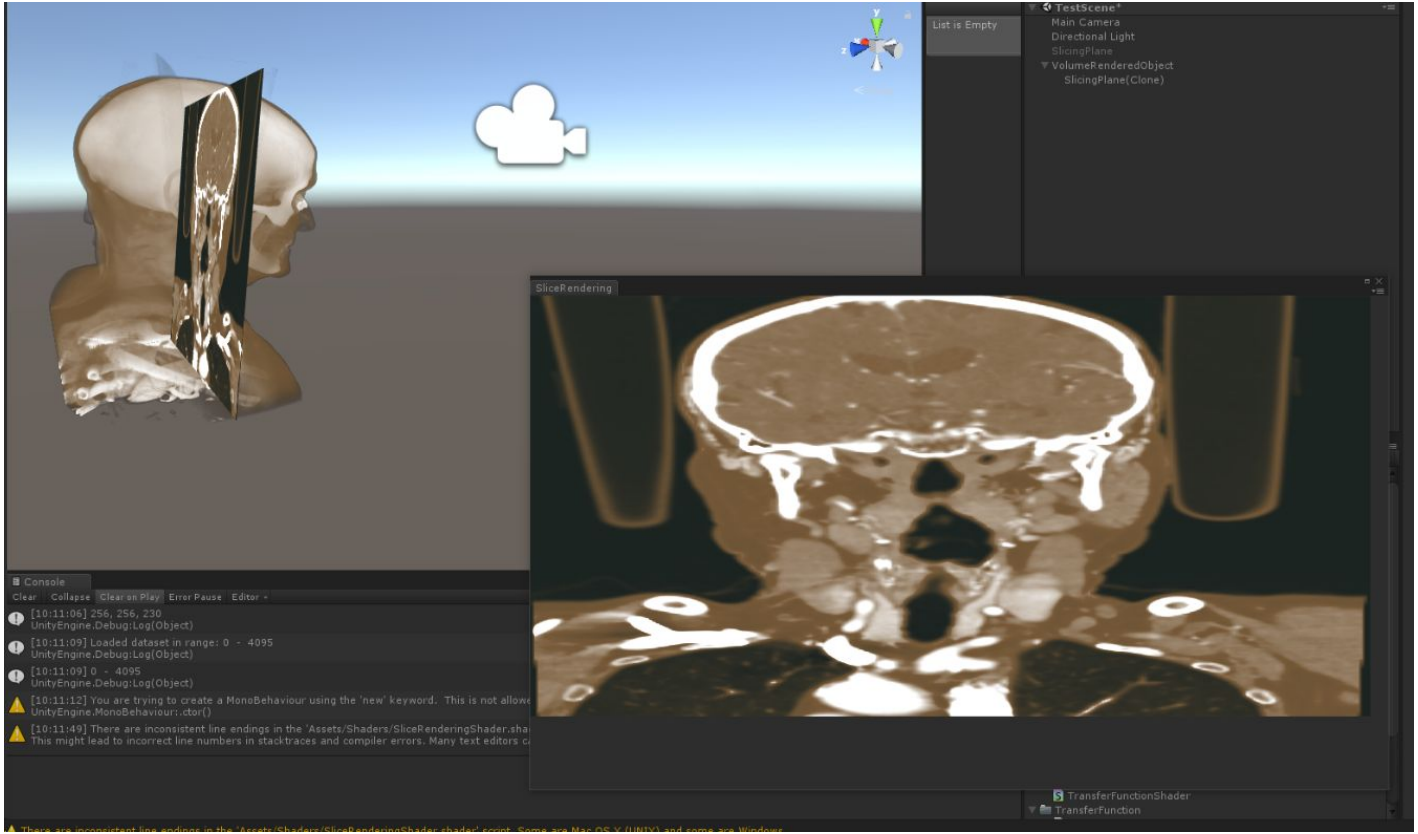

TestScene
Main Camera
Shader
VolumeRendering/TransferFunction2DShader
TransferFunctio
Console
Clear
Collapse
Clear on Play
Error Pause
Editor
[12:39:20] 256, 256, 230
UnityEngine.Debug:Log(Object)
[12:39:24] Loaded dataset in range: 0 - 4095
UnityEngine.Debug:Log(Object)
[12:39:24] 0 - 4095
UnityEngine.Debug:Log(Object)
[12:39:29] You are trying to create a MonoBehaviour using the 'new' keyword. This is not allowed. MonoBehavio
UnityEngine.MonoBehaviour:.ctor()
0.267
0.726
0
0.056
0.8
0.474
Add rectangle

Slice rendering

- Create a movable plane
- In the shader, use the vertex coordinates (relative to the volume model) as 3D texture coordinates.
- (optionally) use 1D transfer function to apply colour

```
o.relVert = mul(_parentInverseMat, float4(vert, 1.0f));
```

```
fixed4 frag (v2f i) : SV_Target
{
    float3 dataCoord = i.relVert +float3(0.5f, 0.5f, 0.5f);
    float dataVal = tex3D(_DataTex, dataCoord).a;
    float4 col = tex2D(_TFTex, float2(dataVal, 0.0f));
    col.a = 1.0f;

    return col;
}
```


TestScene*
Main Camera
Directional Light
SlicingPlane
VolumeRenderedObject
SlicingPlane(Clone)
List is Empty
SliceRendering
Console
Clear
Collapse
Clear on Play
Error Pause
Editor
[10:11:06] 256, 256, 230
UnityEngine.Debug:Log(Object)
[10:11:09] Loaded dataset in range: 0 - 4095
UnityEngine.Debug:Log(Object)
[10:11:09] 0 - 4095
UnityEngine.Debug:Log(Object)
[10:11:12] You are trying to create a MonoBehaviour using the 'new' keyword. This is not allowe
UnityEngine.MonoBehaviour:.ctor()
[10:11:49] There are inconsistent line endings in the 'Assets/Shaders/SliceRenderingShader.sha
This might lead to incorrect line numbers in stacktraces and compiler errors. Many text editors ca
TransferFunctionShader
TransferFunction

References / credits

- ImageVis3D
  - Some of the GUI are heavily inspired by ImageVis3D
  - Get ImageVis3D here: http://www.sci.utah.edu/software/imagevis3d.html
- Marcelo Lima
  - Marcelo presented his implementation of isosurface rendering during his assignment presentation for the Visualisation course at the University of Bergen.
  - See his amazing project here: https://github.com/m-lima/KATscans
- Thanks to Stefan Bruckner and Helwig Hauser who taught me CG and visualisation at the University of Bergen.